Granite — Proposal Defense

Speaking Notes, Timing Plan & Q&A Coaching Guide

Built from Granite_Proposal_Defense_v2.pptx (10 slides) — every line traceable to a slide or its notes
Team: Shahoud Shahid (23i-2515) · Muhammad Noor (23i-2520) · Saif Shahzad (23i-2634)
Supervisor: Dr. Adil Majeed · F26-060-Granite · Stream A

This document is a rehearsal tool, not a script to read verbatim. Read it several times, then speak from memory in your own words — the panel is testing whether you understand the project, not whether you can recite a paragraph. Total target time: **9 minutes 50 seconds**, leaving a natural buffer inside the 10-minute limit.

1. Presentation Overview & 10-Minute Timing Plan

Ten slides, three presenters, one continuous story: what's broken today → why it's genuinely hard → what we built and proved → how we'll prove the rest → who owns what. The timing below is deliberately under 10 minutes so normal delivery speed — including a breath between slides — still lands inside the limit.

#	Slide title	Presenter	Target time	Running total
1	Title & One-Line Problem	Shahoud	~25s	0:25
2	Problem & Users	Shahoud	~55s	1:20
3	Existing Solutions & Prior FYP Comparison	Shahoud	~65s	2:25
4	Research & Technical Basis	Shahoud	~55s	3:20
—	Transition: Shahoud → Noor	—	~15s	3:35
5	Complex Computing Challenge (CCP)	Noor	~65s	4:40
6	Proposed Solution & Contribution	Noor	~65s	5:45
7	POC / Feasibility Evidence	Noor	~65s	6:50
—	Transition: Noor → Saif	—	~15s	7:05
8	Evaluation Plan & Risks	Saif	~60s	8:05
9	Scope, Work Division & Iteration Plan	Saif	~60s	9:05
10	Team Responsibilities, DEI & Ethics/Legal	Saif	~45s	9:50

Total: 9 minutes 50 seconds against a 10-minute limit — a real, usable ~10-second buffer, not a razor's edge. If a presenter runs slightly long on one slide, the next presenter can trim their own opening line rather than rushing the whole remaining deck.

2. Presenter / Slide Allocation

Presenter	Slides owned	Total time	Why this grouping
Shahoud (Data / Benchmark)	1, 2, 3, 4	~3 min 20s	Opens the story: title → problem → what already exists and the one prior FYP checked → the literature that shaped our decisions. Natural fit — the prior-FYP archive check and dataset/benchmark choices are Shahoud's own subsystem.
Noor (Diagnosis Engine)	5, 6, 7	~3 min 15s	Carries the technical core: why this is genuinely hard (CCP), what's actually being built (the diagnosis architecture), and the real proof-of-concept evidence — almost all of which comes from the diagnosis pipeline Noor owns.
Saif (Platform)	8, 9, 10	~2 min 45s	Closes with how it will be judged (evaluation + risks), how the work is split and scheduled, and the team/ethics wrap-up — a natural closing role for the person who owns deployment and the product itself.

3. Complete Slide-by-Slide Speaking Notes

Every number, name and claim below appears on the slide or in its embedded speaker notes. Nothing here introduces a new fact — it explains, in plain spoken words, what the slide already says.

SLIDE 1 — Title & One-Line Problem

SHAHOUD · ~25s

SAY THIS:

Good morning. We're Team Granite. I'm Shahoud, and with me are Noor and Saif. Granite is a self-healing multi-agent system for semantic error diagnosis, verified repair, and recovery.

In one sentence: when a team of AI agents fails — even when the wrong answer looks completely right — no existing method can reliably say which agent caused it, or check whether a proposed fix actually works before you trust it. That's the gap we're closing, under the supervision of Dr. Adil Majeed.

Covers: C.16 slide 1 — title, team, supervisor, stream, one-line problem

SLIDE 2 — Problem & Users

SHAHOUD · ~55s

SAY THIS:

So who actually has this problem? Picture an engineering team that's running several AI "agents" together — think of them as specialist workers on a task: one plans, one looks things up, one writes the final answer. When that team is running in or near production, product teams downstream depend on its output, and auditors need to be able to see what went wrong when it fails.

Here's today's actual workflow, without Granite: a trace of what the agents did gets captured — by tools like LangSmith, Langfuse, Braintrust or Arize — and then a human just reads through it and guesses the root cause. There's no published accuracy for that guess. There's no check at all for what we call a "silent failure" — a confident, wrong answer that doesn't trigger any error or crash, so it slips through unnoticed. And there's no way to test a fix before you ship it.

The field's own measured number for "a human reads the trace and guesses" is 14.2% step accuracy — that's the baseline the entire project is trying to beat. This connects to UN Sustainable Development Goal 9, industry and infrastructure, because this is genuinely new reliability tooling for a genuinely new category of software.

Covers: C.16 slide 2 — stakeholders, current workflow, consequence of failure, SDG

SLIDE 3 — Existing Solutions & Prior FYP Comparison

SHAHOUD · ~65s

SAY THIS:

Is this gap real, or did we just miss a feature that already exists somewhere? We checked, carefully. ErrorProbe, the strongest published academic method, reaches 59.5% step accuracy — but by its own admission it cannot catch silent failures, and it's never been tested on an agent framework outside the two it was built and tuned on.

On the commercial side, LangSmith Engine and Arize Alyx both launched automated diagnosis features this year — we say that upfront, because an early draft of our own research claimed "nobody automates this," and a direct check proved that stale. We corrected it ourselves, on the record. But neither of them publishes an accuracy number, neither is confirmed to test a fix before applying it, and both are locked to their own platform.

And we didn't stop at commercial products — we checked our own department's history too. We went through 660 project titles across every cohort back to 2019 and compared them structurally, not just by

title. The closest one we found is a Spring 2024 project on hallucination detection in a single AI model's text — same underlying problem, but it only covers roughly one of our four pipeline stages, with no multi-agent tracing and no repair step at all.

So the real gap isn't any single missing feature. It's that nothing — not one paper, not one product, not one prior project here — combines attribution, silent-failure detection, verified repair, and generalizing across different agent frameworks, in one working pipeline.

Covers: C.10 — product/prior-FYP/workflow comparison, structural not title-level; C.16 slide 3

SLIDE 4 — Research & Technical Basis

SHAHOUD · ~55s

SAY THIS:

We read seven papers in full, not just abstracts, and every major decision we made traces back to one of them. The field's own progress looks like a climb: a single AI model just guessing scores 14.2%, an early automated method called AgenTracer lands around 37 to 43%, and ErrorProbe — our chosen baseline — reaches 59.5%. We picked ErrorProbe because it's the strongest published number, not because it's the newest.

Three decisions followed directly from that reading. We committed to CrewAI and LangGraph as our two agent frameworks specifically because none of the seven papers we reviewed used either one — that's what actually tests generalization. We chose a rule-based, constraint-checking approach for catching silent failures instead of asking another AI model to judge the output, because the literature shows that approach barely beats a coin flip. And a market survey recommended we narrow down to just one agent framework to keep the project small — we read that advice and declined it, because narrowing to one framework would delete the entire cross-framework question we're trying to answer.

One honest caveat, said before anyone has to ask for it: different papers use different underlying AI models, so some of the reported gap partly reflects raw model strength, not just method quality — we control for that inside our own experiments.

Covers: Section 11/C.11 — research-to-decision table; literature review; research gap

TRANSITION — Shahoud to Noor: "That's the landscape and the gap. Noor will now walk you through why this is genuinely hard, what we're actually building, and the real evidence we already have on disk."

SLIDE 5 — Complex Computing Challenge (CCP)

NOOR · ~65s

SAY THIS:

Before I explain what we're building, I want to answer the question every panel asks first: is this actually hard enough to count as a final-year capstone, or is it routine coding? The handbook gives nine specific characteristics of genuinely complex computing work, and we checked ourselves against every one of them. We clear eight of the nine clearly — things like conflicting requirements, no obvious existing solution, needing real depth of knowledge, and consequences that actually matter if we get it wrong. The ninth, having genuinely diverse stakeholders, we're honestly marking as still developing, because live practitioner contact is scheduled for later in the project, not done yet — we'd rather say that plainly than overclaim it.

But here's the real test the handbook itself proposes: could a capable student, with AI assistance, build this in a week? Our honest answer is yes — for a stripped-down wrapper that just captures a trace and asks a model to guess. But that quick version would get wrong exactly the parts that actually carry the marks: it wouldn't be tested on an unfamiliar agent framework, it couldn't tell a silent failure from a

correct answer, it wouldn't measure a false-positive rate on clean, working examples, and it would have zero proof that any suggested fix actually works. Our real difficulty lives in the data, the evaluation design, and the constraints — not in the wrapper around a model call.

Covers: C.7 — CCP/complex-computing justification, 9 Seoul Accord characteristics, resilience test

SLIDE 6 — Proposed Solution & Contribution

NOOR · ~65s

SAY THIS:

So what are we actually building? The best published method tops out at 59.5% accuracy, has never been tested on an agent framework outside its own benchmark, and admits it misses silent failures. We're building two things on top of that: a new benchmark that tests generalization across different agent frameworks, and a diagnosis pipeline that adds a rule-based check specifically for silent failures — packaged as an installable toolkit, not just a research script.

Here's how it actually works, end to end. The target system — built on CrewAI or LangGraph — runs, and our system watches it. Lane A, which I'll call the data lane, instruments the framework and injects controlled test faults to build our benchmark. Lane B, the diagnosis lane that I personally own, normalizes the execution trace, builds a dependency graph out of it, and runs two things at once: a backward trace to find the responsible agent and step, and our Semantic Error Diagnostic — a rule-based checker that catches confident-but-wrong outputs directly, without ever calling an AI model to judge itself. And where there's genuinely no rule to check against, it says so honestly instead of guessing — that abstention rate is reported as its own number, not hidden. Lane C, the platform lane, then generates a candidate repair, verifies it through a controlled replay before trusting it, and surfaces everything through a dashboard and an installable SDK.

The AI model itself only sits at about fifteen to twenty percent of the components — the differentiating logic, the part that actually makes the diagnosis, never calls it at all.

Covers: C.6 — proposed solution, architecture, contribution statement; C.14 — external component classification

SLIDE 7 — POC / Feasibility Evidence

NOOR · ~65s

SAY THIS:

The handbook requires us to show evidence, not just describe it — so here it is, and I want to lead with the honest negative first. One of our eight proof-of-concept spikes — a preventive classifier — came back at 0.25 precision, recall and F1, essentially random. We're not hiding that; the guide itself says a spike that disproves an assumption is worth more than one that just confirms what you already believed, and every number on this slide is real and recomputable straight from files in our repository — happy to open one live if asked.

Now the real results. Across five independent runs, our reproduction of ErrorProbe's method hit an average of 71.4% agent-level accuracy and 13.2% step-level accuracy — reported as a range because we're honest about run-to-run variance, not just one lucky number. Our Semantic Error Diagnostic caught a genuinely tricky near-miss — twenty dollars reported against nineteen seventy-five actually correct — with zero false alarms. Our repair-and-replay mechanism took a corrupted result of ninety-seven point two and correctly recovered it to one-oh-eight. And our Repair Agent succeeded on four out of six real test cases, with both failures traced to one single, fixable cause. Eight spikes, eight with real evidence on disk. We also have a working, clickable prototype console you can see right here, running against real spike data.

Covers: C.12 — mandatory proposal-stage POC, demonstrated with evidence

TRANSITION — Noor to Saif: "That's the evidence that this already works at small scale. Saif will now cover how we'll prove it at full scale, how the work is split across the team, and wrap up with our timeline and ethics commitments."

SLIDE 8 — Evaluation Plan & Risks

SAIF · ~60s

SAY THIS:

Good evidence isn't enough on its own — you also need to know, in advance, what counts as success. So before running the full reproduction, we pre-registered our acceptance rule: we accept it if it lands within ten points of ErrorProbe's published 59.5%, on a fairly comparable model. We measure agent and step accuracy, how far an error propagates, silent-failure precision and recall, and — something no paper we reviewed reports — a false-positive rate measured on clean, correctly-working runs, not just failing ones.

We also defined four honest outcome bands in advance, each tied to a specific risk. If the reproduction misses our ten-point band, that's reported transparently side-by-side with the published number, exactly as the handbook itself allows — that's the floor, not a failure of the project. If our Recovery Controller mechanism doesn't pass its checkpoint at Mid-1, we fall back to a method we've already proven works, and that lands us at an acceptable result. If our repair memory feature fails its re-test, it still ships as a legitimate before-and-after comparison study, and a strong result means beating the baseline on frameworks it's never seen, with over seventy percent silent-failure accuracy. An exceptional result adds over fifty percent recovery on verified repairs. Every one of these rules was written before we have the results — nothing gets adjusted afterward to make a number look better.

Covers: C.13 — evaluation plan, baseline, metrics, success criteria, failure cases; §5.3 pre-registration

SLIDE 9 — Scope, Work Division & Iteration Plan

SAIF · ~60s

SAY THIS:

Here's exactly who owns what, and why each piece genuinely needs to exist. Shahoud owns data and the benchmark — instrumenting the agent frameworks, injecting test faults, and building and releasing the benchmark itself. Without that piece, there's no cross-framework benchmark and no way to test our core claim at all. Noor owns the diagnosis engine — the graph construction, the ErrorProbe reproduction, the Semantic Error Diagnostic, and the causal tracing. Without that, there's no attribution and no silent-failure detection — the actual contribution disappears. I own the platform — the repair-and-replay engine, the SDK, the backend, and the deployment. Without that piece, nothing here is actually installable outside our own development machines.

Our committed core is eight mechanisms already backed by real evidence, plus two more that are gated on specific criteria we'll test at Mid-1 — eight further extensions only get built once that core genuinely works. The whole project runs at zero committed cost — every model call runs on a free, local model — and we deliberately chose not to spend roughly three-hundred-fifty to eight-hundred-eighty dollars on a paid alternative, reporting that trade-off honestly rather than hiding it. We're scheduled against five formal approval gates from this proposal defense through to final delivery, against a budget of eleven-hundred-thirty planned hours across both semesters — which is exactly what our combined capacity actually is.

Covers: C.5 — scope, ownership, ability to lose a member and still lose a real piece of contribution; timeline

SLIDE 10 — Team Responsibilities, DEI & Ethics/Legal

SAIF · ~45s

SAY THIS:

To close: while each of us owns a subsystem, the architecture, integration, quality, security, data handling, and this defense itself stay shared across all three of us, at every one of our five gates. On responsible development — releasing our benchmark and tools openly widens who can access this kind of tooling, in line with that same Sustainable Development Goal we opened with. Every proposed repair stays advisory, gated behind a human, never applied automatically. We only inject test faults into systems that have actually consented to it. And when a result came back negative, like that classifier spike, we published it as found rather than burying it.

One more thing we want to say plainly, because the handbook asks us to name it directly: our AI usage is Level 2, AI-assisted. We lead every judgment call and every piece of execution ourselves; AI assists on isolated tasks, and it's logged, session by session, for all nineteen sessions so far. Any one of us can explain, justify, and modify our own piece of this system under direct questioning — happy to demonstrate that live, on any part you'd like to pick. Thank you — we're ready for your questions.

Covers: C.8 — AI-assisted development level named and justified; C.16 slide 10 — DEI, ethics, legal, ownership test

4. Presenter Transition Lines (quick reference)

The same two lines already embedded above, collected here for a fast run-through during rehearsal without paging back through the full notes.

> *1 → Shahoud to Noor (after slide 4): "That's the landscape and the gap. Noor will now walk you through why this is genuinely hard, what we're actually building, and the real evidence we already have on disk."*

> *2 → Noor to Saif (after slide 7): "That's the evidence that this already works at small scale. Saif will now cover how we'll prove it at full scale, how the work is split across the team, and wrap up with our timeline and ethics commitments."*

Delivery tip: say the transition while making eye contact with the next presenter, and physically step back or hand across a clicker if you have one — a visible handoff reads as a rehearsed, confident team, not three separate solo presentations stitched together.

5. Proposal-Defense Coverage Checklist

Every area the handbook expects a proposal defense to cover, mapped to exactly where it's said out loud. Nothing on this list is forced in artificially — categories genuinely not applicable at proposal stage (e.g. final deployment evidence, owed at Final-2) are marked as such rather than stretched to fit.

Area	Covered on	How
Problem statement	Slides 1, 2	One-line problem (S1); stakeholders, current workflow, consequence (S2)
Motivation	Slides 2, 5	SDG 9 framing; why the failure mode matters (missed silent failures ship undetected)
Proposed solution	Slide 6	Contribution statement, three-lane architecture, pipeline walkthrough
Objectives / scope	Slides 6, 9	In-scope 10 mechanisms; explicit out-of-scope list; committed core vs. gated vs. extensions
Target users	Slide 2	Primary: engineering teams running multi-agent systems; secondary: product teams, auditors
Key features	Slides 6, 7	Attribution, silent-failure detection, verified repair, cross-framework SDK — each with POC evidence
How the system works	Slide 6	Trace → graph → parallel attribution/SED → repair → verified replay → dashboard/SDK
Methodology / technical approach	Slides 4, 6, 8	Literature-driven decisions; architecture; statistical evaluation protocol
Literature review	Slide 4	7 papers read in full; accuracy lineage 14.2% → ~40% → 59.5%
Research gap	Slide 3	Structural comparison table; "the real gap" stated as a combination, not one feature
Dataset / data sources	Slide 8	Who&When (184 cases), own benchmark, GSM8K + tau-bench, clean controls
Model / algorithm / technology choices	Slides 4, 9	ErrorProbe as baseline; constraint-based SED over LLM-judge; stack listed on S9
Expected results	Slide 8	Pre-registered acceptance band; 4 success bands (Failure/Acceptable/Strong/Exceptional)
Final product	Slide 6	Docker + pip-installable SDK — "operable by someone who didn't build it"
Innovation / novelty	Slide 3, 6	Combination no other system/paper/prior FYP has: attribution + SED + verified repair + cross-arch
Comparison with existing solutions	Slide 3	ErrorProbe, LangSmith Engine/Arize Alyx, Langfuse family, closest prior FYP — one table
Feasibility	Slide 5, 7	9-characteristic CCP self-rating; the "could AI build this in a week" resilience test; real POC evidence
Cost / budget	Slide 9	$0 committed (local model); paid alternative (~$353–$881) priced and declined, on record
Work division among team members	Slides 9, 10	Per-member ownership + "if removed" test; shared responsibilities named separately
Timeline	Slide 9	5 gates (Proposal → Mid-1 → Final-1 → Mid-2 → Final-2); 1,130-hour effort budget
Risks and limitations	Slides 5, 8	Characteristic 6 held honestly weak; 4 named risks each with a stated fallback
DEI statement	Slide 10	Open-source access equity; colour-plus-text accessibility; English-only scope named honestly

Area	Covered on	How
Ethical / legal compliance	Slide 10	Advisory-only repairs; consent for real trace data; MIT/CC BY 4.0 licensing confirmed
AI-assisted development disclosure	Slide 10	Level 2 named and justified per handbook C.8; logged per session, Form 13
Prior-FYP / departmental comparison	Slide 3	660 titles across every cohort since 2019, checked structurally — closed, not left blank

6. Comprehensive Q&A Preparation

Organized from the questions every panel opens with, through to the ones designed to end a weak defense. Rehearse the challenging section hardest — those are the ones a panel actually uses to decide.

6.1 Basic / Expected Questions

Q: What is the problem you're solving?

When AI agents work together and one of them fails, current tools can't reliably say which agent caused it — and none of them can catch a wrong answer that looks completely correct, or check whether a proposed fix would actually work before you trust it.

Key point to remember: Lead with the sentence on slide 1 — it's built to answer this in one breath.

Q: Why is this problem important?

Multi-agent AI systems are already moving into real production use — customer support, coding assistants, research tools. A silent failure in one of these ships a wrong answer with zero warning sign. That's not a hypothetical; it's the specific failure mode current tools cannot catch.

Key point to remember: Anchor to SDG 9 and "ships a confident, wrong answer — no crash, no signal" from slide 2.

Q: Why did you choose this topic?

We read the recent literature on multi-agent failure diagnosis and found a specific, confirmed gap: the strongest published method admits it can't catch silent failures and has never been tested on an agent framework outside its own benchmark. That's a real, evidenced opening, not a topic we picked because it sounded interesting.

Key point to remember: Point to the accuracy lineage (14.2% → ~40% → 59.5%) as proof this is an active, fast-moving area.

Q: What exactly are you building?

A diagnosis-and-repair layer that sits on top of a multi-agent system: it finds which agent and step caused a failure, catches failures that show no error signal at all, generates and tests a fix before trusting it, and ships as an installable SDK, not a one-off script.

Key point to remember: Say "layer on top of," not "replacement for" — it complements existing trace-capture tools.

Q: Who benefits from this, specifically?

The primary user is an engineering team running a multi-agent system in or near production. Downstream, product teams benefit from fewer undetected wrong answers reaching users, and auditors benefit from traceable evidence of what happened and why.

Key point to remember: Three named beneficiaries on slide 2 — primary and two secondary.

Q: What makes your solution different from just asking an AI model what went wrong?

Pasting a trace into a chatbot gives you one unbenchmarked opinion with no ground truth. Ours is a fault-injection harness that creates cases where the true cause is known by construction, so accuracy is a measured number; a rule-based checker that catches silent failures with zero model calls; and a replay mechanism that actually tests a fix before anyone trusts it.

Key point to remember: The field's own baseline for "just ask a model" is 14.2% — that's the number a chatbot answer loses to by 45 points.

6.2 Technical / Methodology Questions

Q: What is your research gap, precisely?

Not one missing feature — a missing combination. No paper, product, or prior project we checked combines published step-level attribution accuracy, general silent-failure detection, verified counterfactual fix-testing, and cross-framework generalization, all in one evaluated pipeline.

Key point to remember: Say "the gap is the combination" — this is the single line to repeat if pressed.

Q: Why did you choose ErrorProbe as your baseline instead of a newer method?

We chose it on measured performance, not recency — it's the strongest published step-accuracy number in the literature we reviewed, at 59.5%.

Key point to remember: "On performance, not recency" is the exact phrase from slide 4 — use it.

Q: Why CrewAI and LangGraph specifically?

Because neither was used by any of the seven papers we reviewed. Testing on frameworks the method was never tuned on is the actual test of whether it generalizes — that's the whole point of the claim.

Key point to remember: This is also why a market survey's advice to narrow to one framework was explicitly declined.

Q: Why a rule-based checker instead of using an AI model to judge outputs?

The literature shows an AI model judging another AI model's output for this kind of silent failure performs barely above chance — the reported figure is an AUROC of about 0.65. A deterministic, rule-based check is falsifiable and auditable in a way an AI-judging-AI approach isn't.

Key point to remember: "Barely above chance" is the phrase to use — it pre-empts "why not just use a bigger model."

Q: What datasets are you using, and why these?

Who&When for baseline reproduction (184 real cases, already in hand), plus our own new benchmark built on GSM8K and tau-bench, including deliberately clean, non-broken control cases so we can measure a false-positive rate — something no paper we reviewed reports.

Key point to remember: Naming the false-positive-rate gap unprompted is a strong, specific point most panels won't expect.

Q: How will you evaluate the system, concretely?

Agent and step accuracy, how far an error spreads through the agents, silent-failure precision and recall, false-positive rate on clean runs, task completion and repair success rates, and latency overhead — measured with bootstrap confidence intervals, fixed seeds, and repeated replays, not a single run.

Key point to remember: All five metric groups are listed on slide 8 — recite the list, don't paraphrase loosely.

Q: What are your success metrics, exactly?

Four pre-registered bands. Failure means missing our accuracy band with no improvement anywhere. Acceptable means at least parity with the published baseline plus a cross-architecture result. Strong means beating the baseline with over 70% silent-failure accuracy. Exceptional adds over 50% verified-repair recovery on top of Strong.

Key point to remember: These were written before any result exists — say that explicitly; it's the whole point.

Q: How will multi-agent systems actually be diagnosed and repaired, step by step?

The target system runs, we capture its trace, build a dependency graph from it, then run two checks in parallel: a backward trace to find the responsible agent and step, and our constraint checker for silent failures. Once a cause is found, a repair candidate is generated and re-executed under a controlled replay before it's ever reported as verified.

Key point to remember: "In parallel" is important — the silent-failure check never waits on the attribution result.

Q: What happens if the repair doesn't actually fix the problem?

It's never applied automatically in the first place — every repair is advisory and gated behind a human. The controlled replay measures a recovery rate across repeated runs before anything is even proposed as trustworthy, and if it fails, that's reported honestly, not hidden.

Key point to remember: "Advisory, human-gated" ties directly to the ethics statement on slide 10.

6.3 Challenging / Skeptical Questions

Q: Your title says self-healing and recovery — show me the evidence. Isn't one of these components unbuilt and the other already failed?

Fair question, and we'd rather say it ourselves than have it found. Recovery Controller has no proof-of-concept yet and Verified Repair Memory's one real test did fail — both carry pre-registered gates at Mid-1, decided before any result exists, each with a stated fallback that still keeps the loop closed either way. Six of our other core mechanisms already have real evidence behind them.

Key point to remember: Lead with this unprompted, don't wait to be asked — concealment reads far worse than an honest gate.

Q: This looks like three separate projects — reproduction, a new benchmark, and a product — for three people in one year. Isn't that too much?

They're one pipeline, not three projects — the benchmark feeds the diagnosis engine, and the SDK is the packaging of that same engine. The budget is real and specific: 1,130 planned hours against 1,130 hours of actual combined capacity, scheduled across five gates.

Key point to remember: Cite the exact hour figure — a specific number beats a reassurance every time.

Q: Your reproduction only reached 13.2% step accuracy versus ErrorProbe's published 59.5%. Isn't your whole comparison built on a baseline you can't even reproduce?

That number is from a deliberately minimal early spike — a simplified version, run five independent times specifically to prove the approach produces real signal before spending more effort, and it did: 71.4% agent accuracy on average. The full reproduction is scheduled with a pre-registered ten-point acceptance band, and if it lands outside that band, both numbers get reported side by side, exactly as the handbook allows for a faithful reproduction. Either outcome is a real result — only staying silent about a gap would be a failure.

Key point to remember: Never say "we'll get a better number" — say "either outcome is reported honestly."

Q: LangSmith and Arize already shipped automated diagnosis this year, built by teams far bigger than yours. Aren't you just rebuilding a funded product?

We checked both directly and corrected our own earlier positioning when we found them — that's on our own record. What neither confirms, as documented capability: a published accuracy number on any benchmark, general silent-failure detection, counterfactual fix-testing with a measured recovery rate, or anything that isn't locked to their own platform. That combination, measured and published, is what we're actually contributing.

Key point to remember: "We checked and corrected our own claim" lands better than defending the original assumption.

Q: Rule-based detection of "semantic" errors sounds like a contradiction — either your rules only check surface formatting, which is trivial, or you're claiming rules understand meaning, which is impossible. Which is it?

Neither extreme. We scope the claim precisely: where a task's own specification yields a checkable constraint — a numeric total that must match, a policy rule that must hold — a deterministic check catches violations of it, including a genuinely subtle near-miss we caught in testing. Where no such constraint exists, the system abstains rather than guessing, and we report that abstention rate as its own metric instead of hiding it.

Key point to remember: "Abstains and reports it" is the exact answer that defuses this — memorize the phrase.

Q: Couldn't a capable student with AI assistance build this whole thing in a couple of weeks?

A stripped wrapper — capture a trace, ask a model what went wrong — yes, in about a week. What that version would get wrong is exactly what carries the marks: no benchmark tested on an unfamiliar framework, no way to distinguish a silent failure from a correct answer, no measured false-positive rate, and zero proof any suggested fix works. That's where our actual difficulty sits.

Key point to remember: This is verbatim the handbook's own resilience test (C.7) — answer it in the handbook's own terms.

Q: You're evaluating your own system against a benchmark you built yourselves. Isn't that grading your own homework?

Ground truth constructed by controlled fault injection — where you already know the true origin point because you injected it — is the field's own established method; ErrorProbe and AgenTracer both do the same thing. What makes it realistic is that the underlying tasks come from unmodified public sources — Who&When, GSM8K, tau-bench — not tasks we wrote ourselves to be easy to solve.

Key point to remember: Name the two datasets and stress "unmodified" — that's what makes it not self-serving.

Q: What if, six months from now, someone publishes a paper that does exactly what you're proposing?

A real, named risk we track openly, checked at every project gate. If it happens, we'd report it honestly as a finding rather than hide it — and the benchmark and the deployable tool we build stay valuable either way, since a single competing paper wouldn't replace either.

Key point to remember: Don't claim it can't happen — claim you're watching for it and have a real answer if it does.

Q: What's the weakest part of your entire proposal, in your own words?

Two committed mechanisms — Recovery Controller and Verified Repair Memory — carry the thinnest evidence: one has no proof-of-concept at all yet, and the other's one real test failed. We named both ourselves, gave each a pre-registered pass/fail test at Mid-1, and a recorded fallback either way, rather than waiting for a panel to find them.

Key point to remember: Answering this unprompted, specifically, is worth more credibility than any slide.

6.4 "Do You Actually Understand Your Own System" Questions

Q: Explain, without notes, exactly what your own subsystem does and what breaks if it's removed.

Each team member should be able to answer this live, unaided, using the exact "if removed" line already written for their own lane on slide 9 — Shahoud: no cross-architecture benchmark, Noor: no attribution algorithm or silent-failure detection, Saif: nothing installable outside the development machine.

Key point to remember: This is a live spoken test, not a document question — the only way to prepare is to rehearse it out loud, alone, from memory.

Q: If I asked you to swap out the underlying AI model for a different one, what would actually break?

Almost nothing that matters to our core claim. Roughly 80–85% of the components — the graph construction, the causal tracing, the constraint checking, the confidence scoring — never call an AI model at all, so they're byte-identical either way. Only the agent's own behavior, the repair generation, and the replay's re-execution step touch a model.

Key point to remember: This is the "replacement test" from the handbook's external-component rule — know the ~15–20% figure cold.

Q: Why is the ability to say a repair 'recovered' the system meaningfully different from just re-running it and hoping?

Because the correction is re-executed under a controlled protocol — everything before the fix stays frozen, decoding settings are pinned — repeated multiple times specifically because model output isn't perfectly repeatable, and the result is a measured recovery rate across those runs, not a single lucky pass.

Key point to remember: Say "controlled, not deterministic" unprompted — it's the exact phrase the team already flagged as needing to be said before being asked.

7. Final Last-Minute Defense Tips

- Lead with weaknesses you already know about — the two gated Tier-2 mechanisms, characteristic 6 held honestly weak, Spike 7's negative result. Saying it first reads as rigor; having it found reads as concealment.
- When a number appears on a slide, know which spike or document it came from and offer to open it live. Every number in this deck is real and traceable — use that as a strength, not just a fact.
- Rehearse the two transition lines out loud with the next presenter until the handoff feels natural, not read. A visible, confident handoff signals a genuinely integrated team.
- Don't read slide 5's nine CCP characteristics one by one — point at the grid and speak the bottom summary band instead, exactly as the slide's own notes say.
- If asked something outside your own slides, it's fine to say "that's [teammate]'s subsystem — [name], can you take that?" A clean handoff is stronger than a guessed answer.
- Time yourself at home at least twice, out loud, with a visible clock. The written timing plan is a guide, not a guarantee — your own natural pace is what actually counts on the day.
- If a question challenges something genuinely unresolved (stakeholder contact, the two Tier-2 gates), answer with the real, current status — never imply something is done when it isn't.
- Breathe before answering a hard question. A three-second pause reads as thinking; a rushed, defensive answer reads as unprepared.

Good luck, Team Granite. Every claim in this document traces back to a real file in your own repository — that's the actual advantage walking into the room.